Deliverable — Reading Scudo's `memtag.h` (T4)

I read the actual Scudo `memtag.h` source (the LLVM copy Android builds from). This confirms several things the guide marked unverified, and answers "how does Scudo use MTE" at the instruction level. Caveat: this is `main`, not a pinned Android release — for the real audit (Phase 0) pin the exact commit for your target build. Mechanisms are stable; constants can move.

In plain English first

Scudo is Android's `malloc` — the code that hands out memory. MTE is a hardware feature that stamps a tiny color on each piece of memory and crashes the program if a pointer's color doesn't match. **The point of this tab:** MTE is just the hardware check; *Scudo is the code that actually chooses the colors and stamps them.* I read Scudo's real source to confirm exactly how it does that — and to settle whether it also uses PA (it doesn't). The three things that matter: tag 0 is always skipped (so odds are 1-in-15, not 1-in-16), there's a switch that turns checks *off* for speed (an audit target), and there is zero Pointer Authentication in the allocator.

Word bank (terms in this tab)

Term	Plain meaning
Allocator	The library behind `malloc` / `free` that hands out and reclaims memory. Scudo is Android's.
Tag / color	The 4-bit value MTE stamps on memory and pointers. "Color" is just my nickname for it.
Granule	The 16-byte unit of memory that gets one tag.
`IRG` / `STG` / `STZG`	ARM instructions: pick a random tag / stamp a tag / stamp-and-zero. Scudo calls these.
PSTATE.TCO	A CPU switch that temporarily turns tag-checking **off**. Scudo flips it in a few spots for speed.

Term	Plain meaning
PA / PAC	Pointer Authentication — a *separate* feature that signs pointers. Confirmed: **not used by Scudo.**
Pinned commit	A specific frozen version of the source. We must pin one for the real audit, since code changes over time.
`prctl` / HWCAP2	Linux calls that check "does this CPU support MTE?" and "turn MTE on for this process."

Confirmed: tag 0 is always excluded

In `selectRandomTag`, the first line is literally `ExcludeMask |= 1`, with the comment "Always exclude Tag 0." So the random tag is drawn from 15 values, not 16.

This is why Project Zero and the AOSP docs say 1/15, not 1/16. **Use 1/15 (≈6.7%)** as the per-attempt collision chance against a random tag.

Confirmed: check suppression via PSTATE.TCO

`ScopedDisableMemoryTagChecks` is a C++ scope guard: on construction it reads the current TCO bit and sets `msr tco, #1` (tag-check-override on); on destruction it restores. While alive, tag checks are off for that thread.

The concrete audit target: any path that runs inside one of these scopes performs memory accesses with no tag checking. The question for an attacker/agent: **can attacker-influenced data be processed inside such a scope?** Greppable: every construction of `ScopedDisableMemoryTagChecks`, and what runs in its lifetime.

How the tagging actually works

The core is `setRandomTag(Ptr, Size, ExcludeMask, ...)`:

1. **Pick the tag:** `selectRandomTag` runs `IRG`, inserting a hardware-chosen random tag into the pointer's top byte, honoring the exclude mask (always includes tag 0).
2. **Paint the memory:** `storeTags(Begin, End)` writes that tag across the allocation. Fast path: `STZG` (store-tag-and-zero, one granule) to a cache-line boundary, then `DC GZVA` (tag-and-zero a whole cache line) for the bulk, then `STZG` for the tail. So allocation

memory is tagged *and zeroed* together. Slow path (plain `STZG` loop) when the region is small or `DC` is unavailable.

Other primitives:

- `extractTag(Ptr)` = `(Ptr >> 56) & 0xf` — pull the 4-bit tag from bits 56–59.
- `untagPointer(Ptr)` = `Ptr & ((1<<56)-1)` — strip the tag.
- `addFixedTag(Ptr, Tag)` — set a specific tag (deterministic, not random).
- `storeTag` (single `STG`), `loadTag` (`LDG`).
- Granule size hard-coded to 16 bytes; minimum alignment ⩾ one granule.

Mental model confirmed: Scudo picks a random non-zero tag with `IRG`, paints it over the allocation (zeroing as it goes) with `STZG` / `DC GZVA`, returns the tagged pointer, retags on free. Hardware checks; Scudo does all policy.

Confirmed: no Pointer Authentication in this file

There is **no PA, no PAC, no `PAC` / `AUT` instruction anywhere in `memtag.h`.** Tagging only. This settles the open question: Scudo's MTE path does not itself use PA. So when Marius says "how Scudo integrates MTE and PA," the PA part is almost certainly *not* inside the allocator's tagging code. It's most likely:

- compiler-generated return-address signing (RET PAC) in the binaries that call Scudo — and Project Zero's Pixel 10 post confirmed Pixel 10 uses RET PAC in place of the stack protector; or
- a separate platform layer; or
- something Marius wants to *add* or evaluate.

The sharp question for Marius: "I read Scudo's memtag.h; it's MTE-only, no PAC. So when we say Scudo integrates PA, do you mean compiler RET PAC in the callers, or a layer you have in mind?"

How an app opts in (partial T6)

`systemSupportsMemoryTagging()` reads the `HWCAP2_MTE` bit from `getauxval`; `enableSystemMemoryTaggingTestOnly()` calls `prctl(PR_SET_TAGGED_ADDR_CTRL, ...)` with `PR_MTE_TCF_SYNC` and a tag-exclusion mask of `0xfffe` (again excludes tag 0, bit 0). The app-facing opt-in (manifest `memtagMode`, etc.) sits above this libc-level machinery.

What this changes

- **Use 1/15, not 1/16**, for random-tag collision probability.
- **The TCO-suppression audit is concrete:** grep the target for `ScopedDisableMemoryTagChecks`.
- **The PA question is now precise** and worth leading with.
- Allocation memory is zeroed as it's tagged, so uninitialized-memory bugs behave differently under MTE — note when choosing a CVE whose primitive depends on stale heap contents.